

AWS Secure Cloud Lab – Phase 1 & 2 (Infrastructure, Security Monitoring, Network Segmentation)

OBJECTIVE

Build a secure, SOC 2–aligned AWS cloud environment from scratch, demonstrating core security architecture concepts including identity governance, network segmentation, encryption, logging, monitoring, and secure access patterns (bastion, NAT, private DB, Kali monitor).

This lab lays the foundation for future expansion into web application testing, SIEM integration, Terraform automation, multi-tier web application, background processing microservices, message security, pentesting, IAM controls and security hardening.

GOALS

- Create secure baseline IAM, MFA, logging, and compliance monitoring
- Design segmented VPC architecture with public and private subnets
- Enable secure inbound access via Bastion (jump host) only
- Launch isolated Web, Database, and Kali monitoring EC2 instances
- Configure NAT Gateway for private subnet outbound-only traffic
- Enable encryption, activity logging, and threat/vulnerability detection
- Deploy basic ALB in multi-AZ configuration for future scaling
- Prepare AWS environment for Phase 3: Terraform, SIEM, Pentesting

TOOLS & SERVICES USED

Identity / Governance:

- IAM Users, MFA, IAM Policies, SSM Parameter Store, KMS

Security Monitoring & Compliance:

- CloudTrail
- AWS Config
- GuardDuty
- Amazon Inspector
- Security Hub

Network / Infrastructure:

- AWS VPC
- Subnets (Public / Private / Multi-AZ)
- Internet Gateway
- NAT Gateway
- Route Tables
- Security Groups

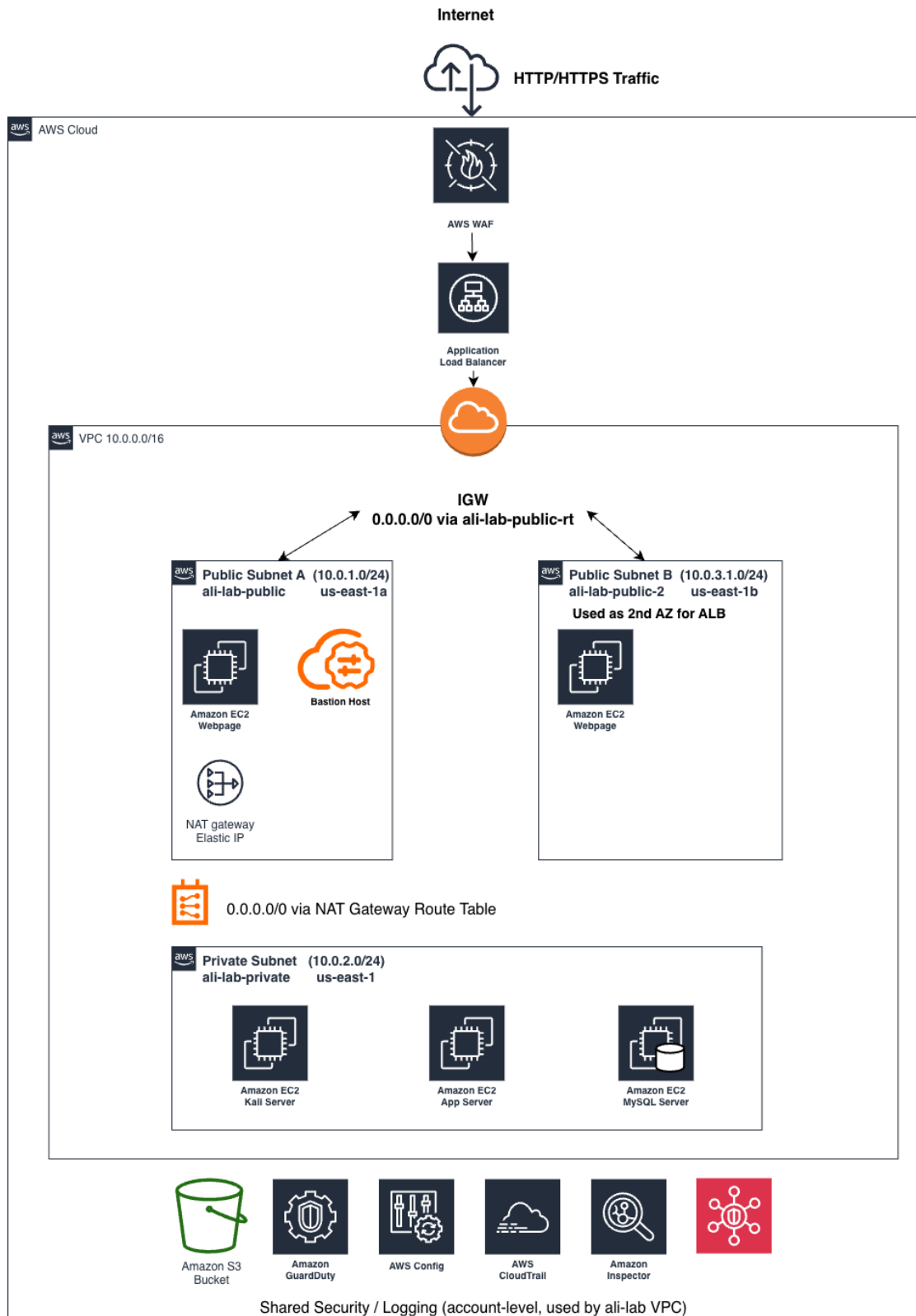
Compute & OS-Level Tools:

- EC2 (Ubuntu Web Server / MySQL DB / Kali Security Box)
- Bash / SSH / MySQL / tcpdump / nmap / hydra / metasploit

Networking Tools Installed (Jump Host & Kali):

- traceroute, wget, curl, dig, tcpdump, ss, nmap, mysql-client
- hydra, gobuster, nikto

HIGH-LEVEL ARCHITECTURE



IMPLEMENTATION

PHASE 1: AWS SECURITY FOUNDATION

Created an Admin User (me) in order to have a baseline access to management tools.
-Created IAM Admin User (not using Root)

IAM > Users > Create user

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Specify user details

User details

User name

Ali-Admin

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and +, -, @, _ (hyphen)

☒ Provide user access to the AWS Management Console - optional

If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Console password

☐ Autogenerated password

You can view the password after you create the user.

☒ Custom password

* Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols ! @ # \$ % ^ & * () _ + - (hyphen) = [] { } '

☐ Show password

☒ Users must create a new password at next sign-in - Recommended

Users automatically get the [IAMUserChangePassword](#) policy to allow them to change their own password.

If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.

Learn more

Cancel

Next

Step 1

Specify user details

Step 2

Set permissions

Step 3

Review and create

Step 4

Retrieve password

Review and create

Review your choices. After you create the user, you can view and download the autogenerated password, if enabled.

User details

User name

Ali-Admin

Console password type

Custom password

Require password reset

Yes

Permissions summary

< 1 >

Name	Type	Used as
Admin	Group	Permissions group
IAMUserChangePassword	AWS managed	Permissions policy

Tags - optional

Tags are key-value pairs you can add to AWS resources to help identify, organize, or search for resources. Choose any tags you want to associate with this user.

No tags associated with the resource.

Add new tag

You can add up to 50 more tags.

Cancel

Previous

Create user

Attached AdministratorAccess IAM policy

Step 1 Specify user details
Step 2 **Set permissions**
Step 3 Review and create
Step 4 Retrieve password

Set permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

- ☒ **Add user to group**
Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.
- ☐ **Copy permissions**
Copy all group memberships, attached managed policies, and inline policies from an existing user.
- ☐ **Attach policies directly**
Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

User groups (1/1)

Search

☒ Group name	☐ Users	☐ Attached policies	☐ Created
☒ Admin	0	AdministratorAccess	2025-11-09 (Now)

[Create group](#)

Set permissions boundary - optional

Set a permissions boundary to control the maximum permissions for this user. Use this advanced feature used to delegate permission management to others. [Learn more](#)

☐ Use a permissions boundary to control the maximum permissions
You can select one of the existing permissions policies to define the boundary.

[Cancel](#) [Previous](#) [Next](#)

Enabled MFA via Google Authenticator

IAM > Users > Create user

Step 1 **Specify user details**
Step 2 Set permissions
Step 3 Review and create
Step 4 Retrieve password

Specify user details

User details

User name
Ali-Admin

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and * _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Console password

- Must be at least 8 characters long
- Must include at least three of the following mix of character types: uppercase letters (A-Z), lowercase letters (a-z), numbers (0-9), and symbols (! @ # \$ % ^ & * () _ - + = { } | ')

☐ Show password

☒ **Users must create a new password at next sign-in - Recommended**
Users automatically get the [MFMUserChangePassword](#) policy to allow them to change their own password.

[If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.](#)
[Learn more](#)

[Cancel](#) [Next](#)

Logged into my admin user that I created and called the ID

```
~ $ aws sts get-caller-identity
{
  "UserId": "AID[REDACTED]",
  "Account": "97[REDACTED]",
  "Arn": "arn:aws:iam::[REDACTED]:user/Ali-Admin"
}
~ $
```

Success

Created S3 logging bucket ([ali-lab-log](#)) and enabled KMS encryption to store centralized logs, config history, and compliance evidence.

- This bucket will later receive services like CloudTrail, Config, and Security Hub) As per SOC2 and ISO 27001 control requirements for log retention and integrity.

```
~ $ aws s3api create-bucket \
> --bucket ali-lab-log \
> --region us-east-1 \
> \
{
  "Location": "/ali-lab-log"
}
```

Creating a managed KMS key, so that we have full control over how this is encrypted.

```

~ $ aws kms create-key \
> --description "CMK for ali-lab-log" \
> --key-usage ENCRYPT_DECRYPT \
> --origin AWS_KMS
{
  "KeyMetadata": {
    "AWSAccountId": "888888888888",
    "KeyId": "88888888-8888-8888-8888-888888888888",
    "Arn": "arn:aws:kms:us-east-1:888888888888:key/88888888-8888-8888-8888-888888888888",
    "CreationDate": "2020-01-01T00:00:00.000Z",
    "Enabled": true,
    "Description": "CMK for ali-lab-log",
    "KeyUsage": "ENCRYPT_DECRYPT",
    "KeyState": "Enabled",
    "Origin": "AWS_KMS",
    "KeyManager": "CUSTOMER",
    "CustomerMasterKeySpec": "SYMMETRIC_DEFAULT",
    "KeySpec": "SYMMETRIC_DEFAULT",
    "EncryptionAlgorithms": [
      "SYMMETRIC_DEFAULT"
    ],
    "MultiRegion": false,
    ...skipping...
  },
  "KeyMetadata": {
    "AWSAccountId": "888888888888",
    "KeyId": "88888888-8888-8888-8888-888888888888",
    "Arn": "arn:aws:kms:us-east-1:888888888888:key/88888888-8888-8888-8888-888888888888",
    "CreationDate": "2020-01-01T00:00:00.000Z",
    "Enabled": true,
    "Description": "CMK for ali-lab-log",
    "KeyUsage": "ENCRYPT_DECRYPT",
    "KeyState": "Enabled",
    "Origin": "AWS_KMS",
    "KeyManager": "CUSTOMER",
    "CustomerMasterKeySpec": "SYMMETRIC_DEFAULT",
    "KeySpec": "SYMMETRIC_DEFAULT",
    "EncryptionAlgorithms": [
      "SYMMETRIC_DEFAULT"
    ],
    "MultiRegion": false,
    "CurrentKeyMaterialId": "88888888-8888-8888-8888-888888888888"
  }
}

```

Creating an Alias for later retrieval.

```

~ $ aws kms list-keys
{
  "Keys": [
    {
      "KeyId": "88888888-8888-8888-8888-888888888888",
      "KeyArn": "arn:aws:kms:us-east-1:888888888888:key/88888888-8888-8888-8888-888888888888"
    }
  ]
}
~ $ aws kms create-alias \
> --alias-name alias/ali-labs \
> --target-key-id 88888888-8888-8888-8888-888888888888
~ $

```

Enabling default encryption for ali-lab-log bucket

- Enabling server side encryption on the s3 bucket using my CMK, this ensures everything hosted on this bucket are automatically encrypted with my KMS key. This will guarantee consistent encryption across this storage bucket.

```

~ $ aws s3api get-bucket-encryption --bucket ali-lab-log
{
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
          "SSEAlgorithm": "aws:kms",
          "KMSEncryptionConfiguration": {
            "KMSMasterKeyID": "alias/ali-labs"
          }
        },
        "BucketKeyEnabled": false
      }
    ]
  }
}
~ $ aws s3api put-bucket-encryption \
> --bucket ali-lab-log \
> --server-side-encryption-configuration '{
>   "Rules": [
>     {
>       "ApplyServerSideEncryptionByDefault": {
>         "SSEAlgorithm": "aws:kms",
>         "KMSEncryptionConfiguration": {
>           "KMSMasterKeyID": "alias/ali-labs"
>         }
>       },
>       "BucketKeyEnabled": true
>     }
>   ]
> }'
~ $ aws s3api get-bucket-encryption --bucket ali-lab-log
{
  "ServerSideEncryptionConfiguration": {
    "Rules": [
      {
        "ApplyServerSideEncryptionByDefault": {
          "SSEAlgorithm": "aws:kms",
          "KMSEncryptionConfiguration": {
            "KMSMasterKeyID": "alias/ali-labs"
          }
        },
        "BucketKeyEnabled": true
      }
    ]
  }
}
~ $

```

Verified, everything looks nominal.

Configure AWS Cloud Trail to Send logs to encrypted S3 Bucket

Enabling CloudTrail to record all API calls and account activity across the environment.

```

~ $ aws cloudtrail create-trail \
> --name ali-lab-trail \
> --s3-bucket-name ali-lab-log \
> --is-multi-region-trail \
> --enable-log-file-validation \
> --kms-key-id alias\ali-labs

An error occurred (InsufficientS3BucketPolicyException) when calling the CreateTrail operation: Incorrect S
~ $

```

Ran into minor error where CloudTrail does not have explicit permission. Well fix it!

```

~ $ aws sts get-caller-identity
{
  "UserId": [REDACTED]
  "Account": [REDACTED]
  "Arn": "arn:aws:iam::[REDACTED]:user/[REDACTED]"
}

~ $ aws kms put-key-policy \
> --key-id [REDACTED] \
> --policy-name [REDACTED] \
> --policy {
>   "Version": "2012-10-17",
>   "Statement": [
>     {
>       "Sid": "Enable IAM User Permissions",
>       "Effect": "Allow",
>       "Principal": {
>         "AWS": [REDACTED]
>       },
>       "Action": "kms:*",
>       "Resource": "*"
>     },
>     {
>       "Sid": "Allow CloudTrail to use the key",
>       "Effect": "Allow",
>       "Principal": {
>         "Service": "cloudtrail.amazonaws.com"
>       },
>       "Action": [
>         "kms:GenerateDataKey*",
>         "kms:Decrypt"
>       ],
>       "Resource": "*",
>       "Condition": {
>         "StringLike": {
>           "kms:": [REDACTED]
>         }
>       }
>     }
>   ]
> }'

~ $ aws cloudtrail create-trail \
> --name ali-lab-trail \
> --s3-bucket-name ali-lab-log \
> --is-multi-region-trail \
> --enable-log-file-validation \
> --kms-key-id alias/ali-labs
{
  "Name": "ali-lab-trail",
  "S3BucketName": "ali-lab-log",
  "IncludeGlobalServiceEvents": true,
  "IsMultiRegionTrail": true,
  "TrailARN": "arn:aws:cloudtrail:us-east-1:[REDACTED]:trail/ali-lab-trail",
  "LogFileValidationEnabled": true,
  "KmsKeyId": "arn:aws:kms:us-east-1:[REDACTED]:alias/ali-labs",
  "IsOrganizationTrail": false
}

~ $

```

Created a script to enable, now let's start logging.

```

~ $ aws cloudtrail start-logging --name ali-lab-trail
~ $ aws cloudtrail get-trail-status --name ali-lab-trail
{
  "IsLogging": true,
  "StartLoggingTime": "2025-11-10T06:36:41.148000+00:00",
  "LatestDeliveryAttemptTime": "",
  "LatestNotificationAttemptTime": "",
  "LatestNotificationAttemptSucceeded": "",
  "LatestDeliveryAttemptSucceeded": "",
  "TimeLoggingStarted": "2025-11-10T06:36:41Z",
  "TimeLoggingStopped": ""
}

```

Verified CloudTrail is successfully logging API activity and delivering logs.

Enabling AWS Config Service

```

~ $ aws configservice start-configuration-recorder \
> --configuration-recorder-name ali-config-recorder

```

Verification

```

~ $ aws configservice describe-configuration-recorder-status
{
  "ConfigurationRecordersStatus": [
    {
      "arn": "arn:aws:config:us-east-1:123456789012:configuration-recorder/ali-config-recorder",
      "name": "ali-config-recorder",
      "lastStartTime": "2025-11-10T06:55:21.669000+00:00",
      "recording": true,
      "lastStatus": "SUCCESS",
      "lastStatusChangeTime": "2025-11-10T06:55:38.742000+00:00"
    }
  ]
}
~ $ aws s3 ls s3://ali-lab-log/AWSLog/Config/
2025-11-10 06:54:46      0 ConfigWritabilityCheckFile

```

[AWS Config](#) > Settings

Settings

Data and delivery

Data retention period and delivery channel settings apply to both the customer managed recorder and all service-linked recorders with paid configuration recorder.

Data retention period
Default period is 7 years

S3 bucket name

ali-lab-log

SNS topic account

-

Customer managed recorder

Service-linked recorders

Recorder

✓ Recording is on

Verification on both ends looks good!

Enabling GuardDuty for continuous threat detection


```
~ $ aws guardduty create-detector --enable
```

Configured to publishing frequency to every 15 minutes

```
~ $ aws guardduty update-detector \
> --detector-id [REDACTED] \
> --enable \
> --finding-publishing-frequency FIFTEEN_MINUTES
```

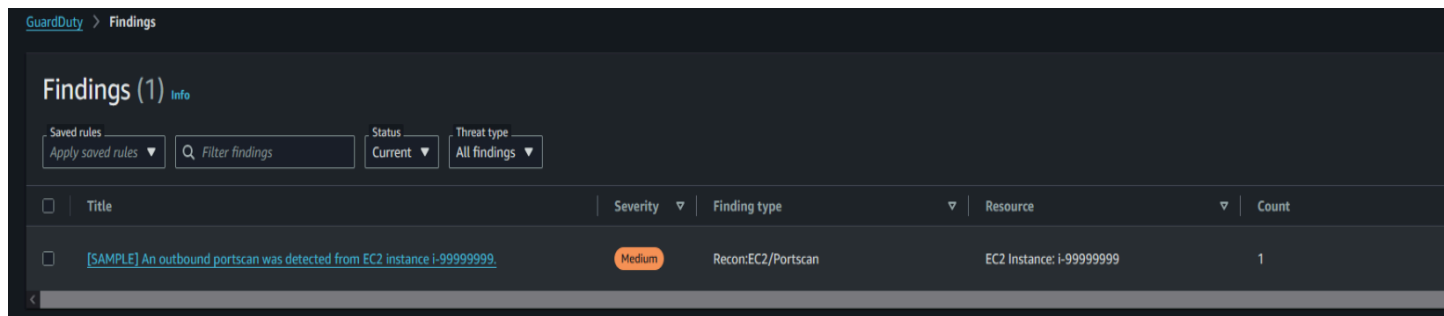
Verified to test if there are any findings (most likely not)

```
~ $ aws guardduty list-findings --detector-id [REDACTED]
{
  "FindingIds": []
}
~ $
```

Seems nominal, let's run a test with a simulated port scan to see if it pops up on our findings and dashboard.

```
~ $ aws guardduty create-sample-findings \
> --detector-id [REDACTED] \
> --finding-types Recon:EC2/Portscan
~ $ aws guardduty list-findings --detector-id [REDACTED]
{
  "FindingIds": [
    "[REDACTED]"
  ]
}
```

Ok, found our sample, Let's see if it comes up in our dashboard.



The screenshot shows the AWS GuardDuty console 'Findings' page. It displays a table with one finding. The finding is titled '[SAMPLE] An outbound portscan was detected from EC2 instance i-99999999'. It has a severity of 'Medium', a finding type of 'Recon:EC2/Portscan', and is associated with the resource 'EC2 Instance: i-99999999'. The count is 1.

Findings (1)	Severity	Finding type	Resource	Count
[SAMPLE] An outbound portscan was detected from EC2 instance i-99999999	Medium	Recon:EC2/Portscan	EC2 Instance: i-99999999	1

Ok, found the sample scan and posted it on our dashboard. Looks great.

Enabling AWS Security Hub

```
~ $ aws securityhub enable-security-hub
```

```
~ $ aws securityhub get-enabled-standards
{
  "StandardsSubscriptions": [
    {
      "StandardsSubscriptionArn": "arn:aws:securityhub:us-east-1:[REDACTED]:subscription/cis-aws-foundational-security-best-practices/v/1.2.0",
      "StandardsArn": "arn:aws:securityhub::ruleset/cis-aws-foundational-security-best-practices/v/1.2.0",
      "StandardsInput": {},
      "StandardsStatus": "PENDING",
      "StandardsControlsUpdatable": "READY_FOR_UPDATES"
    },
    {
      "StandardsSubscriptionArn": "arn:aws:securityhub:us-east-1:[REDACTED]:subscription/aws-foundational-security-best-practices/v/1.0.0",
      "StandardsArn": "arn:aws:securityhub::ruleset/aws-foundational-security-best-practices/v/1.0.0",
      "StandardsInput": {},
      "StandardsStatus": "PENDING",
      "StandardsControlsUpdatable": "READY_FOR_UPDATES"
    }
  ]
}
```

Security Hub Starting up and waiting for provisioning (collecting baseline/initializing control)

```

~ $ aws securityhub get-enabled-standards
{
  "StandardsSubscriptions": [
    {
      "StandardsSubscriptionArn": "arn:aws:securityhub:us-east-1:██████████:subscription/cis-aws-fou
      "StandardsArn": "arn:aws:securityhub:::ruleset/cis-aws-foundations-benchmark/v/1.2.0",
      "StandardsInput": {},
      "StandardsStatus": "READY",
      "StandardsControlsUpdatable": "READY_FOR_UPDATES"
    },
    {
      "StandardsSubscriptionArn": "arn:aws:securityhub:us-east-1:██████████:subscription/aws-foundat
      "StandardsArn": "arn:aws:securityhub:us-east-1::standards/aws-foundational-security-best-practi
      "StandardsInput": {},
      "StandardsStatus": "READY",
      "StandardsControlsUpdatable": "READY_FOR_UPDATES"
    }
  ]
}

```

Waited a couple of moments and they are ready

Enabling Amazon Inspector for Vulnerability Scanning

- Enabling Amazon Inspector to automatically scan EC2 instances, ECR container images, and Lambda Functions for vulnerabilities.

```

~ $ aws inspector2 enable --resource-types EC2 ECR LAMBDA
{
  "accounts": [
    {
      "accountId": "██████████",
      "status": "ENABLING",
      "resourceStatus": {
        "ec2": "ENABLING",
        "ecr": "ENABLING",
        "lambda": "ENABLING",
        "lambdaCode": "DISABLED",
        "codeRepository": "DISABLED"
      }
    }
  ],
  "failedAccounts": []
}

```

All foundational security configurations are up and running.

All foundational security configurations are up and running.

This concludes Phase 1.

We now have:

End-to-end encryption at rest (S3 and KMS)

Multi-region logging with integrity validation (Cloud Trail)

Full configuration history (AWS Config)

Automated threat and vulnerability detection (GuardDuty and Inspector)

Centralized posture management (Security Hub)

Check list:

Service	Purpose	Completion
Log Storage (S3 Bucket)	Encrypted log/evidence storage	✓
KMS	Self-Managed Encryption Keys	✓
CloudTrail	Activity Logging	✓
AWS Config	Activity Logging	✓
GuardDuty	Threat Detection	✓
Security Hub	Posture Management	✓
Inspector2	Vulnerability Scanning	✓

PHASE 2: VPC, SUBNETS, ROUTE TABLES, AND SECURITY GROUPS

Created Custom VPC:

- CIDR: 10.0.0.0/16

```
~ $ aws ec2 create-vpc \
> --cidr-block 10.0.0.0/16 \
> --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=ali-lab}]'
{
  "Vpc": {
    "OwnerId": "arn:aws:iam::123456789012:user:ali-lab",
    "InstanceTenancy": "default",
    "Ipv6CidrBlockAssociationSet": [],
    "CidrBlockAssociationSet": [
      {
        "AssociationId": "vpc-cidr-assoc-12345678",
        "CidrBlock": "10.0.0.0/16",
        "CidrBlockState": {
          "State": "associated"
        }
      }
    ],
    "IsDefault": false,
    "Tags": [
      {
        "Key": "Name",
        "Value": "ali-lab"
      }
    ],
    "VpcId": "vpc-12345678",
    "State": "pending",
    "CidrBlock": "10.0.0.0/16",
    "DhcpOptionsId": "dopt-12345678"
  }
}
~ $ aws ec2 describe-vpcs --vpc-ids vpc-12345678
{
  "Vpcs": [
    {
      "OwnerId": "arn:aws:iam::123456789012:user:ali-lab",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-12345678",
          "CidrBlock": "10.0.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": false,
      "Tags": [
        {
          "Key": "Name",
          "Value": "ali-lab"
        }
      ],
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-12345678",
      "State": "available",
      "CidrBlock": "10.0.0.0/16",
      "DhcpOptionsId": "dopt-12345678"
    }
  ]
}
~ $
```

VPC created and verified

Created Subnets:

- Public Subnet: 10.0.1.0/24 (for Bastion, Web Server, NAT)
- Private Subnet: 10.0.2.0/24 (for DB and Kali)
- Public Subnet 2: 10.0.3.0/24 (for future redundancy and load balancer)

```
~ $ aws ec2 create-subnet \
> --vpc-id vpc-
> --cidr-block 10.0.1.0/24 \
> --availability-zone us-east-1a \
> --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=ali-lab-public}]'
{
  "Subnet": {
    "AvailabilityZoneId": "use1-az1",
    "MapCustomerOwnedIpOnLaunch": false,
    "OwnerId": ,
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
      {
        "Key": "Name",
        "Value": "ali-lab-public"
      }
    ],
    "SubnetArn": "arn:aws:ec2:us-east-1: /subnet-",
    "EnableDns64": false,
    "Ipv6Native": false,
    "PrivateDnsNameOptionsOnLaunch": {
      "HostnameType": "ip-name",
      "EnableResourceNameDnsARecord": false,
      "EnableResourceNameDnsAAAARecord": false
    },
    "SubnetId": "subnet-",
    "State": "available",
    "VpcId": "vpc-",
    "CidrBlock": "10.0.1.0/24",
    "AvailableIpAddressCount": 251,
    "AvailabilityZone": "us-east-1a",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false
  }
}
~ $ aws ec2 create-subnet \
> --vpc-id vpc-
> --cidr-block 10.0.2.0/24 \
> --availability-zone us-east-1a \
> --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=ali-lab-private}]'
{
  "Subnet": {
    "AvailabilityZoneId": "use1-az1",
    "MapCustomerOwnedIpOnLaunch": false,
    "OwnerId": ,
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "Tags": [
      {
        "Key": "Name",
        "Value": "ali-lab-private"
      }
    ],
    "SubnetArn": "arn:aws:ec2:us-east-1: /subnet-",
    "EnableDns64": false,
    "Ipv6Native": false,
    "PrivateDnsNameOptionsOnLaunch": {
      "HostnameType": "ip-name",
      "EnableResourceNameDnsARecord": false,
      "EnableResourceNameDnsAAAARecord": false
    },
    "SubnetId": "subnet-",
    "State": "available",
    "VpcId": "vpc-",
    "CidrBlock": "10.0.2.0/24",
    "AvailableIpAddressCount": 251,
    "AvailabilityZone": "us-east-1a",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false
  }
}
~ $
```

ali-lab-public , designed for public facing resources such as a bastion jump host and web server.

ali-lab-private , designed for internal systems such as a DB server, Kali Security monitoring box and a Linux app server.

Creating Internet Gateway, and attached it to the [ali-lab](#) VPC to enable outbound internet access for public-facing resources.

```
~ $ aws ec2 create-internet-gateway \
> --tag-specifications 'ResourceType=internet-gateway,Tags=[{Key=Name,Value=ali-lab-igw}]'
{
  "InternetGateway": {
    "Attachments": [],
    "InternetGatewayId": "igw-██████████",
    "OwnerId": "██████████",
    "Tags": [
      {
        "Key": "Name",
        "Value": "ali-lab-igw"
      }
    ]
  }
}
```

Attaching gateway to VPC

```
~ $ aws ec2 attach-internet-gateway --internet-gateway-id igw-0f██████████8 --vpc-id vpc-██████████0
~ $ aws ec2 describe-internet-gateways --internet-gateway-ids igw-██████████8
{
  "InternetGateways": [
    {
      "Attachments": [
        {
          "State": "available",
          "VpcId": "vpc-██████████0"
        }
      ],
      "InternetGatewayId": "igw-██████████8",
      "OwnerId": "██████████",
      "Tags": [
        {
          "Key": "Name",
          "Value": "ali-lab-igw"
        }
      ]
    }
  ]
}
```

Double-checking gateway is attached to VPC

```
~ $ aws ec2 describe-internet-gateways
{
  "InternetGateways": [
    {
      "Attachments": [
        {
          "State": "available",
          "VpcId": "vpc-██████████t"
        }
      ],
      "InternetGatewayId": "igw-██████████",
      "OwnerId": "██████████",
      "Tags": []
    },
    {
      "Attachments": [
        {
          "State": "available",
          "VpcId": "vpc-██████████0"
        }
      ],
      "InternetGatewayId": "igw-██████████8",
      "OwnerId": "██████████",
      "Tags": [
        {
          "Key": "Name",
          "Value": "ali-lab-igw"
        }
      ]
    }
  ]
}
```

Creating Route tables and Configuring Public Routing

```
~ $ aws ec2 create-route-table \
> --vpc-id vpc- \
> --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=ali-lab-public-rt}]'

{
  "RouteTable": {
    "Associations": [],
    "PropagatingVgws": [],
    "RouteTableId": "rtb-",
    "Routes": [
      {
        "DestinationCidrBlock": "10.0.0.0/16",
        "GatewayId": "local",
        "Origin": "CreateRouteTable",
        "State": "active"
      }
    ],
    "Tags": [
      {
        "Key": "Name",
        "Value": "ali-lab-public-rt"
      }
    ],
    "VpcId": "vpc-",
    "OwnerId": "97
  },
  "ClientToken": "
}
```

```
~ $ aws ec2 describe-route-tables --filters Name=vpc-id,Values=vpc-
{
  "RouteTables": [
    {
      "Associations": [],
      "PropagatingVgws": [],
      "RouteTableId": "rtb-",
      "Routes": [
        {
          "DestinationCidrBlock": "10.0.0.0/16",
          "GatewayId": "local",
          "Origin": "CreateRouteTable",
          "State": "active"
        },
        {
          "DestinationCidrBlock": "0.0.0.0/0",
          "GatewayId": " ",
          "Origin": "CreateRoute",
          "State": "active"
        }
      ],
      "Tags": [
        {
          "Key": "Name",
          "Value": "ali-lab-public-rt"
        }
      ],
      "VpcId": " ",
      "OwnerId": "
    },
    {
      "Associations": [
        {
          "Main": true,
```

Verified Route table, everything looks good.

Associating the route table With Public Subnet

```
~ $ aws ec2 associate-route-table \  
> --route-table-id rtb-  
> --subnet-id subnet-  
> --region us-east-1
```

Associating route table with Private subnet

```
~ $ aws ec2 associate-route-table --route-table-id --subnet-id subnet-  
{  
  "AssociationId": "rtbassoc-  
  "AssociationState": {  
    "State": "associated"  
  }  
}
```

Creating Security groups and Access Management

Created three security groups within the [ali-lab](#) VPC to enforce network segmentation and access control.

The public SG ([ali-lab-public-sg](#)) allows controlled administrative access (SSH and ICMP) from the internet.

The private SG ([ali-lab-private-sg](#)) restricts traffic to only accept connections originating from the public SG.

This layer implements defense-in-depth and aligns with SOC 2 and ISO 27001 network isolation best practices.

Public security group for jump hosts

```
~ $ aws ec2 create-security-group \  
> --group-name ali-lab-public-sg \  
> --description "Public SG for ali-lab (jump hosts)" \  
> --vpc-id vpc-  
{  
  "GroupId": "  
  "SecurityGroupArn": "arn:aws:ec2:us-east-1:group/sg-
```

Private security group for internal services

```
~ $ aws ec2 create-security-group \  
> --group-name ali-lab-private-sg \  
> --description "Private SG for ali-lab (internal services)" \  
> --vpc-id vpc-  
{  
  "GroupId": "sg-  
  "SecurityGroupArn": "arn:aws:ec2:us-east-1:group/sg-
```

DB

```
~ $ aws ec2 create-security-group --group-name ali-lab-db-sg --description "Database security group for ali lab environment" --vpc- ion us-east-  
{  
  "GroupId": "  
  "SecurityGroupArn": "arn:aws:ec2:us-east-1:
```


Configuring inbound and outbound access rules

Configured inbound rules for both the public and private security groups to control access between subnets and external networks.

- The **public SG** allows SSH and ICMP for remote administration.
- The **private SG** accepts all traffic from the public SG only, ensuring that internal systems remain isolated from the internet.
This configuration enables controlled access and aligns with defense-in-depth and least-privilege security principles.
- The **DB SG**

DescribeSecurityGroups			
Description	ID	Name	VPC
Public SG for ali-lab (jump hosts)	sg-██████████	ali-lab-public-sg	vpc-██████████
Inbound			
From	Proto		To
22	tcp		22
-1	icmp		-1
DescribeSecurityGroups			
Description	ID	Name	VPC
Database security group for ali lab environment	██████████	ali-lab-db-sg	██████████
Inbound			
From	Proto		To
3306	tcp		3306
DescribeSecurityGroups			
Description	ID	Name	VPC
Private SG for ali-lab (nternal services)	sg-██████████	ali-lab-private-sg	vpc-██████████
Inbound			
From	Proto		To
None	-1		None

Creating a key pair for SSH Bastion Jump Host and Securing it

Key Creation

```
~ $ aws ec2 create-key-pair --key-name ali-lab-key --query "KeyMaterial" --output text > ali-lab-key.pem
```

Securing the key

```
~ $ chmod 400 ali-lab-key.pem
```

I applied Linux permissions (`chmod 400`) to restrict access to only the file owner, as required by SSH.

I also securely stored a backup copy in AWS Systems Manager Parameter Store (SecureString), where it is encrypted with KMS and access-controlled via IAM. This ensures secure storage, traceability, and recovery without leaving unprotected copies in CloudShell or local machines.

Storing the key backup onto AWS SSM

```
~ $ aws ssm put-parameter --name "ali-lab-key-private" --type "SecureString" --value "$(cat ali-lab-key.pem)" --description "Private SSH key for ali bastion host" --region us-east-1
{
  "Version": 1,
  "Tier": "Standard"
}
~ $ aws ssm get-parameter \
> --name "ali-lab-key-private" \
> --with-decryption \
> --query "Parameter.Value" \
> --output text > ali-lab-key.pem
bash: ali-lab-key.pem: Permission denied
```

The private SSH key was securely backed up in AWS Systems Manager Parameter Store as a SecureString.

Access to the encrypted key requires explicit `ssm:GetParameter` and `kms:Decrypt` permissions, which are intentionally restricted.

- This aligns with least-privilege IAM, SOC2, and ISO27001 principles for cryptographic key management.

Error with not associating this security group to my VPC (fixed by deleting it and Remaking it) - Shown in troubleshooting selection

Bastion Host Launching

Creating Instance

```
~ $ aws ec2 run-instances \
> --image-id ami-
> --instance-type t2.micro \
> --key-name ali-lab-key \
> --subnet-id subnet-
> --security-group-ids sg-
> --associate-public-ip-address
```

Adding Tag to Bastion Host

```
~ $ aws ec2 create-tags \
> --resources i-
> --tags Key=Name,Value=ali-bastion-host
```

Describing the instance, Everything seems normal

```
-----
| DescribeInstances |
+-----+
| 3.218.247.44 |
+-----+
```

Logged in VIA ssh and everything was successful

```
~ $ ssh -i "ali-lab-key.pem" ubuntu@3.218.247.44
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1012-aws x86_64)
```

When logged onto the Jump host I installed some tools for admin work but wanted to keep it lightweight.

Tools installed: net-tools , iproute2 , traceroute , tcpdump , fail2ban , auditd , iptables , session-manager , nmap , curl , wget , htop , sshpass mysql-client, aws-cli

Configure Private Subnet Access from Bastion Host

We are now allowing the Bastion Host to securely connect to private EC2 instances using SSH. Instead of exposing the private servers to the internet, we restrict their SSH access so it's only allowed from the Bastion Host's security group. This is a core security architecture pattern called **jump host access control**.

Allow SSH from Bastion to Private EC2

```
~ $ aws ec2 authorize-security-group-ingress \
> --group-id sg-████████████████████ \
> --protocol tcp \
> --port 22 \
> --source-group sg-████████████████████ \
> --region us-east-1
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-████████████████████",
      "GroupId": "sg-████████████████████",
      "GroupOwnerId": "████████████████████",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "ReferencedGroupInfo": {
        "GroupId": "sg-████████████████████",
        "UserId": "████████████████████"
      },
      "SecurityGroupRuleArn": "arn:aws:ec2:us-east-1:████████████████████:group-rule/sgr-████████████████████"
    }
  ]
}
```

Creating EC2 Instance on the Private Subnet, Testing Connectivity between hosts

```
~ $ aws ec2 run-instances \
> --image-id ami-████████████████████c \
> --instance-type t2.micro \
> --key-name ali-lab-key \
> --subnet-id subnet-████████████████████37 \
> --security-group-ids sg-████████████████████5 \
> --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=ali-private-ec2}]' \
> --region us-east-1
{
  "Instances": [
    {
      "InstanceId": "i-████████████████████",
      "SubnetId": "subnet-████████████████████37",
      "SecurityGroups": [
        "sg-████████████████████5"
      ],
      "KeyName": "ali-lab-key",
      "ImageId": "ami-████████████████████c",
      "InstanceType": "t2.micro",
      "State": "pending",
      "Placement": {
        "AvailabilityZone": "us-east-1a"
      },
      "Tags": [
        {
          "Key": "Name",
          "Value": "ali-private-ec2"
        }
      ],
      "Monitoring": {
        "Enabled": false
      },
      "NetworkInterfaces": [
        {
          "NetworkInterfaceId": "eni-████████████████████",
          "SubnetId": "subnet-████████████████████37",
          "PrivateIpAddress": "10.0.2.37",
          "PrivateIpAllocationId": "ipalloc-████████████████████",
          "MacAddress": "8a:56:20:00:00:00",
          "DeviceIndex": 0
        }
      ],
      "MetadataOptions": {
        "HttpTokens": "required",
        "HttpEndpoint": "disabled"
      }
    }
  ]
}
```

Transferring keys to the Bastion instance to be able to connect to the Private subnet instances

```
~ $ chmod 400 ali-lab-key.pem
~ $ scp -i ali-lab-key.pem ali-lab-key.pem ubuntu@3.218.247.44:/home/ubuntu/
ali-lab-key.pem
~ $ ssh -i "ali-lab-key.pem" ubuntu@3.218.247.44
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1012-aws x86_64)
```

Connecting from Bastion to private subnet instance

```
ubuntu@ip-10-0-1-154:~$ chmod 400 ali-lab-key.pem
ubuntu@ip-10-0-1-154:~$ ssh -i "ali-lab-key.pem" ubuntu@10.0.2.37
```

Success!

```
ubuntu@ip-10-0-2-37:~$ █
```

INSTANCE CREATION

Creating the public web server on the public subnet

```
~ $ aws ec2 run-instances \  
> --image-id ami-██████████ \  
> --instance-type t2.micro \  
> --key-name ali-lab-key \  
> --subnet-id subnet-0██████████ \  
> --security-group-ids sg-██████████ \  
> --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=ali-web-ec2}]' \  
> --associate-public-ip-address \  
> --region us-east-1
```

Creating Kali instance for security on the private subnet

```
~ $ aws ec2 run-instances \  
> --image-id ami-██████████ \  
> --instance-type t2.micro \  
> --key-name ali-lab-key \  
> --subnet-id subnet-0██████████ \  
> --security-group-ids sg-██████████ \  
> --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=ali-kali-ec2-1}]' \  
> --region us-east-1
```

Creating DB instance on Private subnet

```
~ $ aws ec2 run-instances \  
> --image-id ami-0██████████ \  
> --instance-type t2.micro \  
> --key-name ali-lab-key \  
> --subnet-id subnet-0██████████ \  
> --security-group-ids sg-██████████ \  
> --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=ali-db-ec2}]' \  
> --region us-east-1
```

Allocating an Elastic IP Address

```
~ $ aws ec2 allocate-address --domain vpc --region us-east-1  
{  
  "AllocationId": "eipalloc-██████████",  
  "PublicIpv4Pool": "amazon",  
  "NetworkBorderGroup": "us-east-1",  
  "Domain": "vpc",  
  "PublicIp": "3.225.28.155"
```

```
-----  
|               DescribeAddresses               |  
+-----+-----+-----+  
|   AllocationID   |   PublicIP   |  
+-----+-----+-----+  
| eipalloc-██████████ | 3.225.28.155 |  
+-----+-----+-----+
```

Creating NAT Gateway for Private subnet to allow out bound traffic only on the private subnet

- The NAT Gateway allows EC2 instances in the private subnet to securely access the internet (for system updates, package downloads, external APIs) *without exposing them to the public internet.*

We placed the NAT Gateway in the public subnet and assigned it an Elastic IP, which gives it a consistent public-facing IP address for outbound traffic.

```
~ $ aws ec2 create-nat-gateway \  
> --subnet-id subnet-  
> --allocation-id eipalloc-  
> --tag-specifications 'ResourceType=natgateway,Tags=[{Key=Name,Value=ali-lab-nat-gw}]' \  
> --region us-east-1
```

In this step, I updated the route table associated with the **private subnet** so that all outbound traffic destined for the internet (**0.0.0.0/0**) is routed through the **NAT Gateway** instead of an Internet Gateway.

This allows my private EC2 instances to securely access the internet (for updates, software installation, API calls) **without being publicly exposed**

```
~ $ aws ec2 create-route \  
> --route-table-id rtb-  
> --destination-cidr-block 0.0.0.0/0 \  
> --nat-gateway-id nat-  
> --region us-east-1  
{  
  "Return": true  
}
```

Setting up Services instances (Web Server, DB, and Kali Box)

Setting up database instance

```
ubuntu@ip-10-0-2-207:~$ sudo apt install mysql-server
```

Enabling MySQL

```
ubuntu@ip-10-0-2-207:~$ sudo systemctl enable mysql  
Synchronizing state of mysql.service with SysV service script with /lib/systemd/systemd-sysv-install.  
Executing: /lib/systemd/systemd-sysv-install enable mysql  
ubuntu@ip-10-0-2-207:~$ sudo systemctl start mysql  
ubuntu@ip-10-0-2-207:~$ sudo systemctl status mysql  
• mysql.service - MySQL Community Server  
  Loaded: loaded (/lib/systemd/system/mysql.service; enabled; vendor preset: enabled)  
  Active: active (running) since Mon 2025-11-24 03:57:54 UTC; 5min ago  
    Main PID: 72610 (mysqld)  
      Status: "Server is operational"  
     Tasks: 38 (limit: 1121)  
    Memory: 364.0M  
       CPU: 2.432s  
    CGroup: /system.slice/mysql.service  
            └─72610 /usr/sbin/mysqld
```


Configuring SQL database

```
ubuntu@ip-10-0-2-207:~$ sudo mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.44-0ubuntu0.22.04.1 (Ubuntu)

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE USER 'ali-admin'@'10.0.2.%' IDENTIFIED BY [REDACTED];
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE DATABASE ali_lab_db;
Query OK, 1 row affected (0.01 sec)

mysql> GRANT ALL PRIVILEGES ON ali_lab_db.* TO 'ali-admin'@'10.0.2.%';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)
```

Database instance is **not reachable from the internet**, only from private subnet

MySQL user is **restricted to private subnet IPs**, enforcing segmentation

This lays ground for **app-to-db security and secure future web application testing**

- Supports compliance models (SOC 2, ISO 27001, PCI-DSS) — no public DB access.

Setting Up Kali Security Monitoring Box

```
(kali@kali)-[~]
└─$ sudo apt install nmap netcat-traditional tcpdump nikto hydra wireshark tshark
└─$ sudo apt install -y metasploit-framework
```

Reaching nginx web server

```
~ $ curl http://3.226.245.191
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
  body {
    width: 35em;
    margin: 0 auto;
    font-family: Tahoma, Verdana, Arial, sans-serif;
  }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>
```

The webpage is working and is ready for additional configuration in the future.

Creating a Simple ALB to a fixed response just for practice

Created a duplicate subnet in us-east-1b for fail over and redundancy, in addition to adding AWS Web Application Firewall for an extra layer of added security.

Ali-Lab-ALB

▼ Details

Load balancer type

Application

Scheme

Internet-facing

Status

✓ Active

Hosted zone

VPC

vpc-

Availability Zones

us-east-1b (use1-az2)

us-east-1a (use1-az1)

Load balancer IP address type

IPv4

Date created

November 24, 2025, 13:38 (UTC-05:00)

Load balancer ARN

arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/ali-lab-alb

DNS name

Ali-Lab-ALB-1234567890.us-east-1.elb.amazonaws.com

Also Launched AWS WAF to protect against OWASP Top 10

TROUBLESHOOTING

Modifying Public Subnet to allow assigning public IPs to any instance brought up on the Public Subnet

```
~ $ aws ec2 modify-subnet-attribute --subnet-id subnet-12345678 --map-public-ip-on-launch
```

Issue with security group had to create a new one

```
~ $ aws ec2 create-security-group \
> --group-name db-sg-ali-lab \
> --description "DB security group for lab environment"
{
  "GroupId": "sg-12345678",
  "SecurityGroupArn": "arn:aws:ec2:us-east-1:123456789012:security-group/sg-12345678"
}
```

Deleted it

Tried to connect to web server with public IP associated, no Connection, Adjusted SG to allow inbound HTTP traffic

```
~ $ aws ec2 authorize-security-group-ingress \
> --group-id sg-12345678 \
> --protocol tcp \
> --port 80 \
> --cidr 0.0.0.0/0 \
> --region us-east-1
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sg-rule-12345678",
      "GroupId": "sg-12345678",
      "GroupOwnerId": "123456789012",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "0.0.0.0/0",
      "SecurityGroupRuleArn": "arn:aws:ec2:us-east-1:123456789012:security-group-rule/sg-rule-12345678"
    }
  ]
}
~ $ curl http://3.226.245.191
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

FINAL VALIDATION CHECKPOINT

- Bastion → Private EC2 SSH = Success
- Private EC2 → Internet (via NAT) = Success
- Web Server reachable via browser = Success
- Database isolated from internet = Success
- Kali can conduct internal network assessment = Success
- GuardDuty detects scans = Success

NEXT PHASE (Phase 3 Preview)

- Terraform Infrastructure as Code (VPC, SG, EC2, NAT, IAM)
- Web Application + DB Real Integration
- Deploy Wazuh / Security Onion / Splunk for SIEM
- Penetration testing workflows
- Honeypot and threat intelligence simulation

Conclusion

I intentionally built this environment manually using the AWS CLI to fully understand each networking and security configuration from the ground up. This approach helped me reinforce core AWS concepts such as subnet isolation, secure instance access, NAT and Internet routing, IAM, CloudTrail logging, Security Groups, and encrypted storage.

This VPC now serves as a foundational infrastructure that I plan to expand my knowledge with Terraform, implement security automation, integrate applications, and later build and test secure web services. It represents the baseline architecture for future cloud engineering, and DevSecOps projects.